

Lecture 13 — Looking at Data

Visualization, and what to do when the histogram isn't a bell

Amir Farbin

Part 1 — Visualization

Three levels of data

- **Raw:** images, sound, detector output. Not much to plot — open one and check it is what it claims
- **Tabular:** one row per data point, one column per random variable. *Today's level*
- **Summary statistics:** the output of an analysis of a table. Part 3 comes back to it
- Raw becomes tabular by processing: run algorithms, extract numbers, put them in a table
- Most charts you see in the world live at the third level

Rule zero: look at the file

- Before any library: how big is it, and what is in it?
- `!head` — header row? separator? junk?
- Which columns are categorical, which are numerical — that decides the plot
- This one is clean. Part 3's is not

→ cells 5, 9

Rule zero: look at the file — cell 5

```
!head heart.csv
```

```
age,sex,cp,trestbps,chol,fbs,restecg,thalach,exang,oldpeak,slope,ca,thal,target  
63,1,3,145,233,1,0,150,0,2.3,0,0,1,1  
37,1,2,130,250,0,1,187,0,3.5,0,0,2,1  
41,0,1,130,204,0,0,172,0,1.4,2,0,2,1  
56,1,1,120,236,0,1,178,0,0.8,2,0,2,1  
57,0,0,120,354,0,1,163,1,0.6,2,0,2,1  
57,1,0,140,192,0,1,148,0,0.4,1,0,1,1  
56,0,1,140,294,0,0,153,0,1.3,1,0,2,1  
44,1,1,120,263,0,1,173,0,0,2,0,3,1  
... (1 more lines)
```

Rule zero: look at the file — cell 9

df

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	\
0	63	1	3	145	233	1	0	150	0	2.3	
1	37	1	2	130	250	0	1	187	0	3.5	
2	41	0	1	130	204	0	0	172	0	1.4	
3	56	1	1	120	236	0	1	178	0	0.8	
4	57	0	0	120	354	0	1	163	1	0.6	
..	...	...	..	...	...	...	...	...	...	...	
298	57	0	0	140	241	0	1	123	1	0.2	
299	45	1	3	110	264	0	1	132	0	1.2	
...	(18 more lines)										

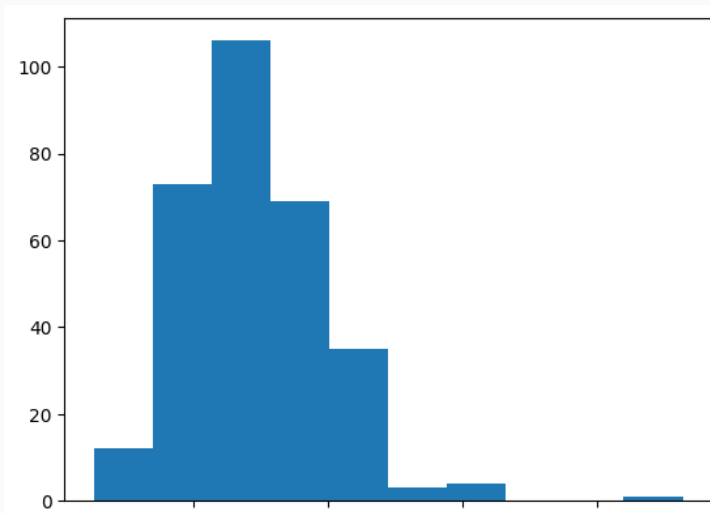
Why histogram first

- Every column is a random variable
- A histogram is the estimate of its probability distribution your sample allows
- Two populations → two distributions → does the variable know which is which?
- Handed a new labelled dataset, with ten seconds: histogram every column for both populations

→ cell 14

Why histogram first — cell 14

```
plt.hist(df["chol"])
```



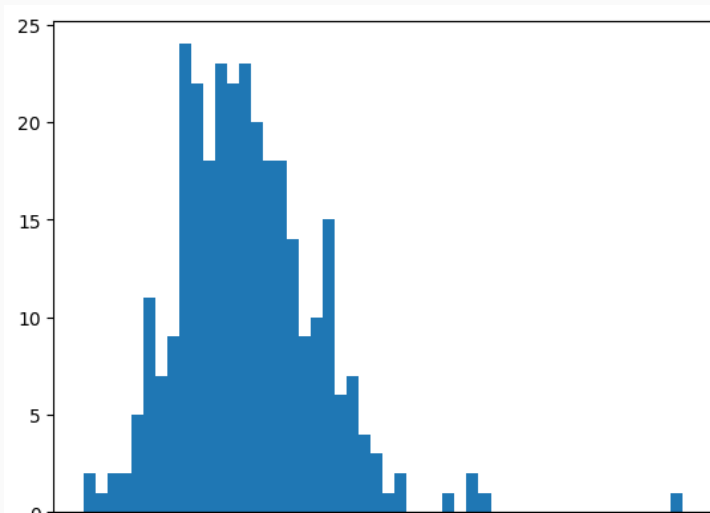
How many bins?

- Ten, because ten is the default. Not because ten is right
- Too wide: structure smeared away, and nothing tells you it happened
- Too narrow: 348 of 500 bins empty, every bump a candidate fluctuation
- As few bins as the question allows, but enough counts per bin to tell two parts apart

→ cells 17, 23

How many bins? — cell 17

```
plt.hist(df["chol"],bins=50)
```



How many bins? — cell 23

```
for bins in [5,10,25,50,100,500]:  
    print("Number of bins:",bins)  
    _=plt.hist(df["chol"],bins=bins)  
    plt.show()  
    #plt.legend()
```



Is that bump real?

- Filling a histogram is a **counting experiment**
- A bin with n entries carries $\sqrt{n}$
- Expect 10, see 14? That is 10 ± 3 — a fluctuation, and it will happen in a few bins out of fifty
- Before calling a bump a sub-population, compare it against the square root of its own contents
- Other boring explanation: one patient entered ten times

When can you see two populations at all?

- Two normals, 2.5σ apart, pretending to be one cholesterol column
- Vary the sample size *and* the binning
- Ten patients: nothing, at any binning
- A hundred: a shoulder, if the binning cooperates
- A thousand and up: unmistakable, over a wide middle range
- More data buys finer bins

→ cell 27

When can you see two populations at all? — cell 27

```
np.random.seed(3402)

n_patients=[10,100,1000,10000]
bin_counts=[10,25,50,100,500]

plt.figure(figsize=(16,12))

plot_i=0
for n in n_patients:
    sample=list(np.random.normal(0.,1.,n//2))+list(np.random.normal(2.5,1.,n//...

    for n_bins in bin_counts:
        plot_i+=1
        plt.subplot(len(n_patients),len(bin_counts),plot_i)
... (7 more lines)
```

Two populations, properly

- Disease vs no disease — a mask per population, `df[mask]`
- 165 and 138 patients
- Four things have to be fixed before the plot means anything

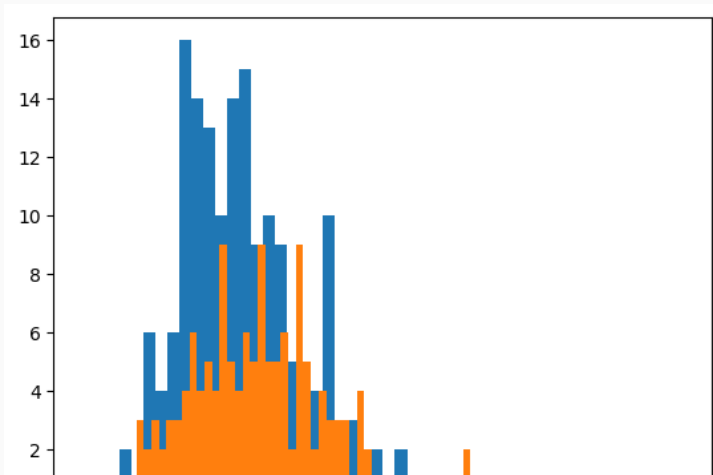
→ cells 30, 34

Two populations, properly — cell 30

```
disease_selection=df["target"]==1  
no_disease_selection=df["target"]==0
```

Two populations, properly — cell 34

```
_ = plt.hist(df[disease_selection]["chol"], bins=50)  
_ = plt.hist(df[no_disease_selection]["chol"], bins=50)
```



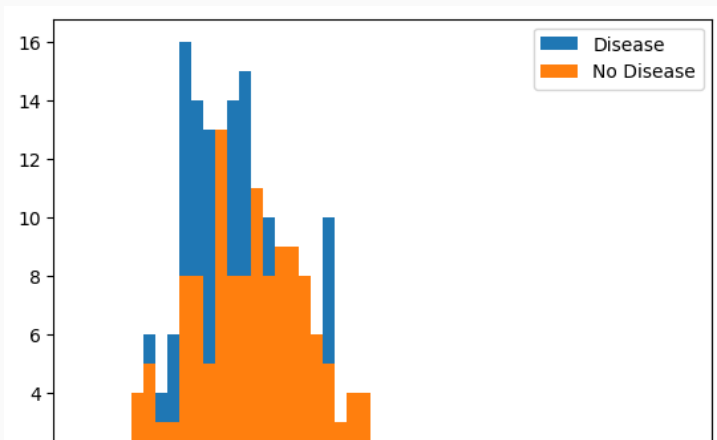
Four fixes, one plot

- **Legend** — which is which
- **Transparency** — the second one hides the first
- **Shared bin edges** — each call picks its own from its own min/max, so bars don't line up
- **Density** — 165 vs 138 patients; counts aren't comparable, areas are
- Result: the two cholesterol distributions sit on top of each other
- That is a finding. Report a null result like one

→ cells 36, 38, 42

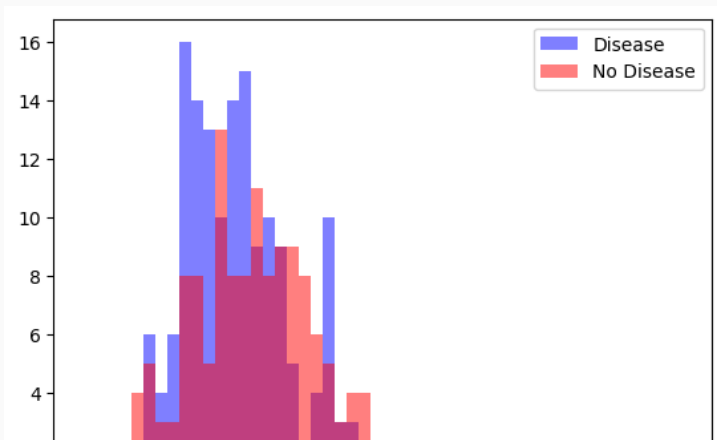
Four fixes, one plot — cell 36

```
_,bins,_=plt.hist(df[disease_selection]["chol"],bins=50,label="Disease")
_=plt.hist(df[no_disease_selection]["chol"],bins=bins,label="No Disease")
_=plt.legend()
```



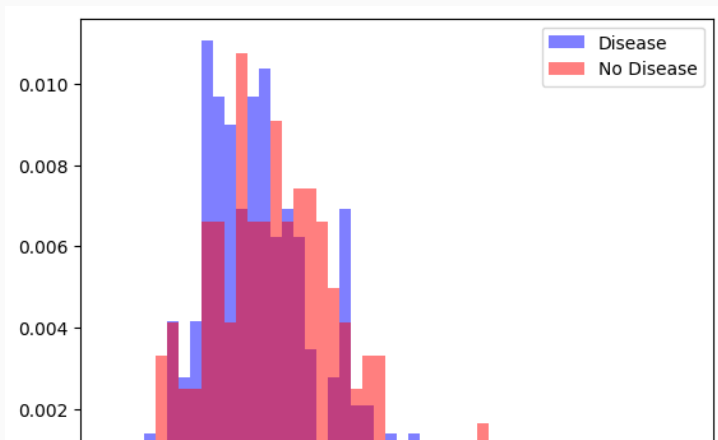
Four fixes, one plot — cell 38

```
_,bins,_=plt.hist(df[disease_selection]["chol"],bins=50,alpha=.5,color='b',lab...  
_=plt.hist(df[no_disease_selection]["chol"],bins=bins,alpha=.5,color='r',label...  
_=plt.legend()
```



Four fixes, one plot — cell 42

```
_,bins,_=plt.hist(df[disease_selection]["chol"],bins=50,alpha=.5,color='b',den...  
_=plt.hist(df[no_disease_selection]["chol"],bins=bins,alpha=.5,color='r',densi...  
_=plt.legend()
```



Write the function

- Five cells to get right, applied to one of fourteen columns
- The cost of asking decides whether the question gets asked
- A ten-minute plot doesn't get made. A one-line plot gets made on a hunch
- `**kwargs` passes options through to `plt.hist` without naming them

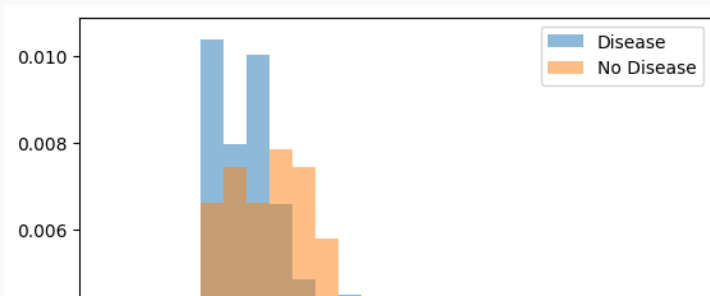
→ cells 47, 50

Write the function — cell 47

```
def compare_distributions(df, column_name, selections, bins=10, **kwargs):  
    for label, selection in selections.items():  
        _, bins, _ = plt.hist(df[selection][column_name], bins=bins, label=label, **k...  
  
    _ = plt.legend()
```

Write the function — cell 50

```
compare_distributions(df,"chol",  
                    {"Disease":df["target"]==1,  
                     "No Disease":df["target"]==0},  
                    alpha=0.5,  
                    density=1,  
                    bins=25,  
                    )
```



Every feature at once

- Fourteen figures you have to scroll are not fourteen figures you can compare
- A 5×3 grid; layout is part of the analysis, not decoration
- target separates perfectly — that panel is a by-product, not a finding

→ cells 54, 59

Every feature at once — cell 54

```
selection_dict={"Disease":df["target"]==1,"No Disease":df["target"]==0}

for column_name in df.columns:
    print(column_name)
    compare_distributions(df,column_name,
                        selection_dict,
                        alpha=0.5,
                        density=1,
                        bins=25,
                        )

plt.show()
```



Every feature at once — cell 59

```
selection_dict={"Disease":df["target"]==1,"No Disease":df["target"]==0}

plt.figure(figsize=(15,15))

for i,column_name in enumerate(df.columns):
    plt.subplot(5,3,i+1)
    compare_distributions(df,column_name,
                        selection_dict,
                        alpha=0.5,
                        density=1,
                        bins=25,
                        stacked=True
                    )
    plt.xlabel(column_name)
... (1 more lines)
```

Where the signal is

- Routine clinical numbers — cholesterol, resting blood pressure, blood sugar — barely differ
- The exercise-test results do: thalach, oldpeak, exang, slope
- Plus cp, ca, and sex
- A few minutes of histogramming, no analysis, and you know where to look

Categorical: a table beats a histogram

- Three or four values = three or four numbers. Bars add nothing
- Drawn with 25 bins, a two-valued variable is actively misleading
- A table has room for the uncertainty, which is what decides if a difference is real

→ cells 62, 67

Categorical: a table beats a histogram — cell 62

```
def compare_categorical(df, column_name, selections, **kwargs):  
    out = list()  
    values = df[column_name].unique()  
  
    for label, selection in selections.items():  
        total = np.sum(selection)  
        df_0=df[selection]  
        out.append([label] + [ np.sum(df_0[column_name]==val)/total for val in...  
  
    return out, values
```

Categorical: a table beats a histogram — cell 67

```
for cat_feature in ["sex","thal","ca", "slope", "exang","restecg", "fbs"]:  
    print(cat_feature)  
    results = compare_categorical(df,cat_feature,selection_dict)  
    display(HTML(tabulate.tabulate(results[0], tablefmt='html', headers=result...
```

sex

<IPython.core.display.HTML object>

thal

<IPython.core.display.HTML object>

ca

<IPython.core.display.HTML object>

... (12 more lines)

The direction that matters

- Normalized by population: *of the patients with the disease, this fraction had each value* — $P(x \mid \text{pop})$
- A diagnosis asks the reverse: *given this value, which population?* — $P(\text{pop} \mid x)$
- You cannot read one off the other without knowing how big each population is
- 83% of the no-disease group is male; and three quarters of the women here have the disease
- Both true. Only the second is about a patient in front of you

→ cell 69

The direction that matters — cell 69

```
def compare_categorical_T(df,column_name,selections,**kwargs):
    out = list()
    values = df[column_name].unique()
    N_val = dict([ (val, np.sum(df[column_name]==val)) for val in values ])

    for label,selection in selections.items():
        total = np.sum(selection)
        df_0=df[selection]
        out.append([label] + [ np.sum(df_0[column_name]==val)/N_val[val] for v...

    return out,values

... (6 more lines)
```

1-D plots are for functions

- A histogram is really zero-dimensional: one variable, counted
- x against y needs one y per x — a function, not a column of data
- Plot two data columns that way and `plot` joins the dots in row order: nonsense
- The one x - y plot you *will* make: training loss against iteration

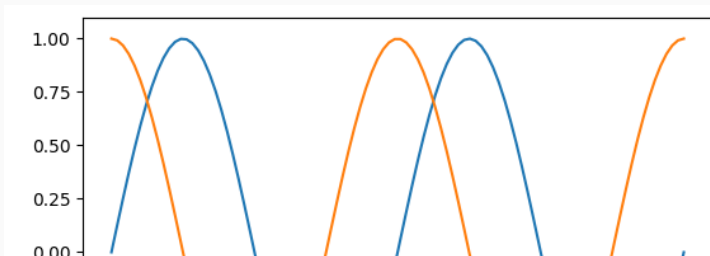
→ cells 73, 76

1-D plots are for functions — cell 73

```
x = np.linspace(-2.*math.pi,2.*math.pi,100)
y_1 = list(map(math.sin,x))
y_2 = list(map(math.cos,x))

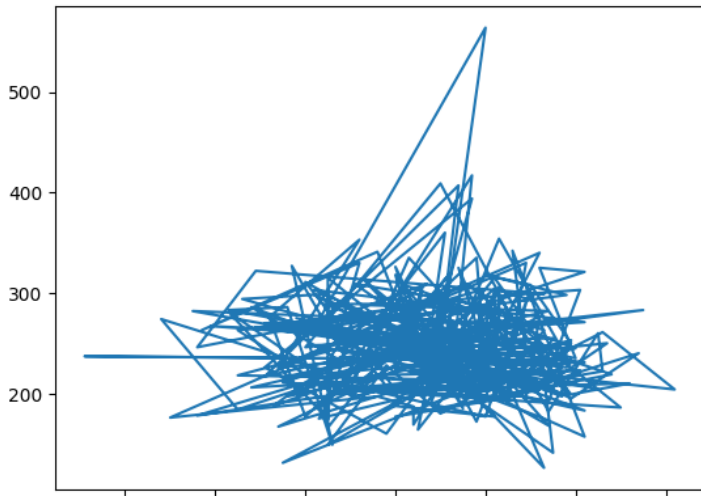
_=plt.plot(x,y_1,label="sin")
_=plt.plot(x,y_2,label="cos")

_=plt.legend()
```



1-D plots are for functions — cell 76

```
_ = plt.plot(df["thalach"], df["chol"])
```



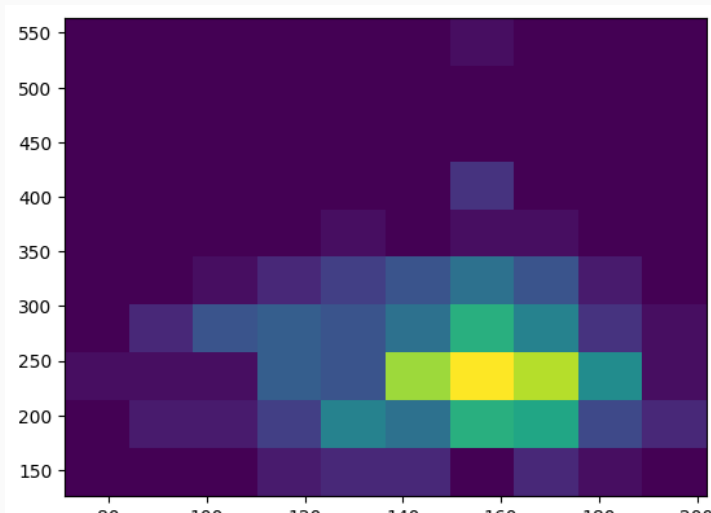
2-D: heat maps and scatter

- Bin both variables, count per box, colour as the third axis
- Hard to read, and impossible to overlay two populations
- Scatter: one marker per patient, nothing joined — your eye does the binning
- Ask: do the two colours sit in different regions? Then a boundary separates them

→ cells 79, 84

2-D: heat maps and scatter — cell 79

```
_ = plt.hist2d(df["thalach"], df["chol"])
```



2-D: heat maps and scatter — cell 84

```
selection_dict={"Disease":df["target"]==1,"No Disease":df["target"]==0}

for label,selection in selection_dict.items():
    _=plt.scatter(df[selection]["thalach"],df[selection]["chol"],label=label)

_=plt.xlabel("thalach")
_=plt.ylabel("chol")

_=plt.legend()
```



The pair plot

- Every pair as a scatter, every diagonal as that variable's histogram
- One figure: all the 1-D comparisons and all the 2-D ones
- **This is the first thing to do with a new dataset**
- It will also show things that are not there — it produces leads, not conclusions

→ cell 94

The pair plot — cell 94

```
# Pair plot
```

```
columns = df.columns[1:6]  
n_columns=len(columns)  
plt.figure(figsize=(15,15))
```

```
plot_i=0
```

```
for i,x_var_name in enumerate(columns):  
    for j,y_var_name in enumerate(columns):  
        plot_i+=1  
        plt.subplot(n_columns,n_columns,plot_i)  
        make_legend = plot_i==1  
        if i==j:  
            compare_distributions(df,x_var_name,  
... (7 more lines)
```

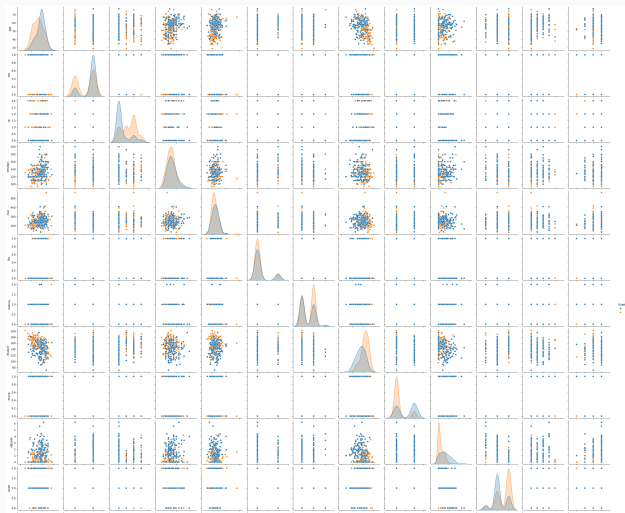
Seaborn: beautiful, and it decides for you

- The diagonal is no longer a histogram — it is a smoothed density estimate
- Everything looks like a tidy bell, whether or not it is one
- Sparse bins, spiky categories, “does this bump survive its own error” — all invisible
- Also draws categoricals as continuous, and it is slow
- Smoothing is a modelling decision. Make it deliberately, or stay on the raw counts

→ cell 98

Seaborn: beautiful, and it decides for you — cell 98

```
sns.pairplot(df, hue="target")
```



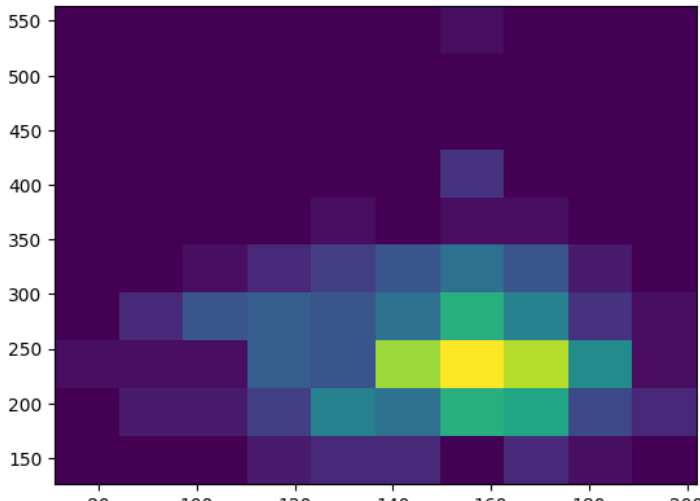
Contour = topographic map

- Same 2-D histogram, drawn a third way
- Lines of equal count, exactly like equal elevation
- Easiest of the three to read — and the standard picture for anything being minimized
- You will meet it again in machine learning

→ cells 101, 102

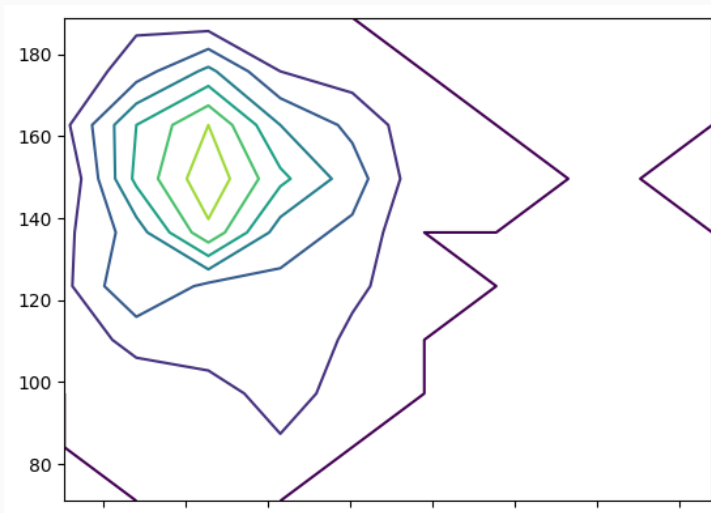
Contour = topographic map — cell 101

```
z,x,y,_=plt.hist2d(df["thalach"],df["chol"])
```



Contour = topographic map — cell 102

```
plt.contour(y[:-1],x[:-1],np.array(z))
```



Part 1, in one procedure

- Normalized distributions of every variable, for each population
- Then scatter plots of the features in pairs
- Then the pair plot that combines them
- The alternative — working before looking — ends in time spent on broken data, or on a question the data could never answer

Part 2 — Does it look normal?

Why we expect a bell

- The **Central Limit Theorem**: add up enough independent pieces and the sum is normal, whatever the pieces look like
- That is why noise, heights, and detector readings are normal
- But “enough independent pieces” hides four assumptions

Four assumptions, four ways to break

- **Identically distributed** — broken by mixing populations
- **Independent** — broken by correlated samples
- **Finite variance** — broken by heavy tails
- **Additive** — broken by pieces that multiply
- Real data guarantees none of them
- This is a *diagnosis*: it doesn't look normal — which one broke, and which plot tells you?

What success looks like

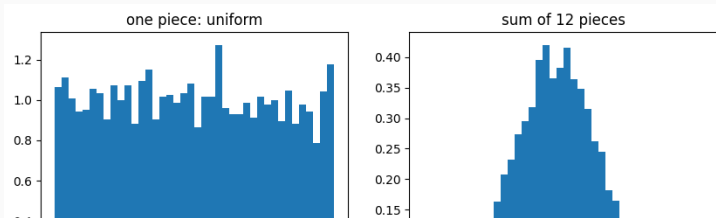
- The sum of 12 uniforms. One uniform is flat; twelve added is a bell
- Mean 6, variance exactly 1 — that is why twelve

→ cell 108

What success looks like — cell 108

```
pieces = np.random.uniform(0, 1, size=(5000, 12))  
sums = pieces.sum(axis=1)
```

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))  
axes[0].hist(pieces[:, 0], bins=40, density=True)  
axes[0].set_title("one piece: uniform")  
axes[1].hist(sums, bins=40, density=True)  
axes[1].set_title("sum of 12 pieces")  
plt.show()
```



Two tools

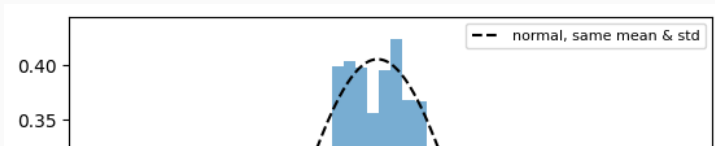
- **Overlay the normal** the data implies: same mean, same standard deviation
- **Q-Q plot:** your quantiles against the normal's. A straight line means normal
- Bends up on the right → right tail. S-shape → both tails. Steps → discreteness

→ cells 112, 115, 118

Two tools — cell 112

```
def hist_with_normal(x, bins=50, ax=None, label=None):  
    if ax is None:  
        ax = plt.gca()  
    x = np.asarray(x)  
    _, edges, _ = ax.hist(x, bins=bins, density=True, alpha=0.6, label=label)  
    grid = np.linspace(edges[0], edges[-1], 300)  
    ax.plot(grid, stats.norm.pdf(grid, np.mean(x), np.std(x)), "k--", label="n...  
    ax.legend(fontsize=8)
```

```
hist_with_normal(sums)
```



Two tools — cell 115

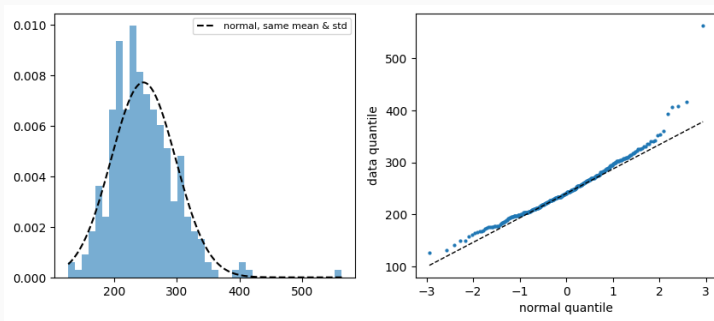
```
def qq_plot(x, ax=None, label=None):
    if ax is None:
        ax = plt.gca()
    x = np.sort(np.asarray(x))
    n = len(x)
    theoretical = stats.norm.ppf((np.arange(1, n + 1) - 0.5) / n)
    ax.scatter(theoretical, x, s=4, label=label)

    q25, q50, q75 = np.percentile(x, [25, 50, 75])
    slope = (q75 - q25) / (stats.norm.ppf(0.75) - stats.norm.ppf(0.25))
    ax.plot(theoretical, q50 + slope * theoretical, "k--", lw=1)
    ax.set_xlabel("normal quantile")
    ax.set_ylabel("data quantile")

... (2 more lines)
```

Two tools — cell 118

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))  
hist_with_normal(df["chol"], bins=40, ax=axes[0])  
qq_plot(df["chol"], ax=axes[1])  
plt.show()
```



\begin{center}

\end{center}

Failure 1: a mixture

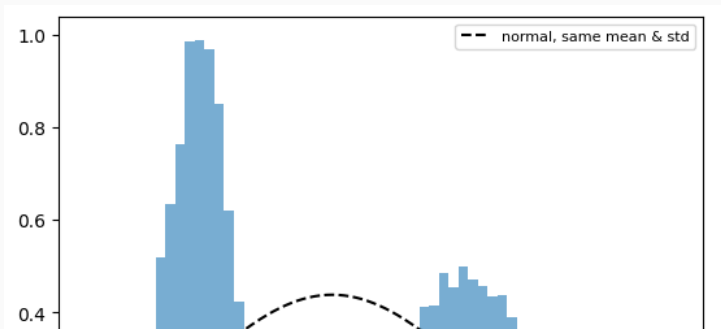
- Two populations in one histogram: mobile and desktop load times
- The pooled variance is *larger* than either group's — the gap between the means contributes
- The fix is the covariate: split by it and the two normals reappear
- How far apart? Below about 2σ you get one lopsided hump, not two

→ cells 120, 126, 130

Failure 1: a mixture — cell 120

```
mobile = np.random.normal(2.5, 0.4, 5000)  
desktop = np.random.normal(0.8, 0.2, 5000)  
load_time = np.concatenate([mobile, desktop])
```

```
hist_with_normal(load_time, bins=60)  
plt.xlabel("page load time (s)")
```



Failure 1: a mixture — cell 126

```
print("pooled :", np.mean(load_time))  
print("mobile :", np.mean(mobile))  
print("desktop:", np.mean(desktop))
```

```
pooled : 1.6519172533576454  
mobile : 2.5033761420127156  
desktop: 0.8004583647025749
```

Failure 1: a mixture — cell 130

```
sigma = 1.0
fig, axes = plt.subplots(1, 4, figsize=(16, 3.5))

for ax, separation in zip(axes, [0.5, 1.0, 2.0, 3.0]):
    group_1 = np.random.normal(0, sigma, 5000)
    group_2 = np.random.normal(separation * sigma, sigma, 5000)
    pooled = np.concatenate([group_1, group_2])
    _, bins, _ = ax.hist(pooled, bins=50, color="gray", alpha=0.4, label="pool...")
    ax.hist(group_1, bins=bins, histtype="step", label="group 1")
    ax.hist(group_2, bins=bins, histtype="step", label="group 2")
    ax.set_title("$\\Delta\\mu$ = " + str(separation) + "$\\sigma$")

axes[0].legend(fontsize=8)
plt.show()
```



Failure 2: correlated samples

- Each value tells you something about the next — AR(1)
- The histogram looks perfectly normal. The **shape tests all pass**
- The order-based panels catch it: a running mean that wanders, autocorrelation that decays slowly
- Your error bars are wrong: the effective sample size is smaller than n

→ cells 135, 138, 140

Failure 2: correlated samples — cell 135

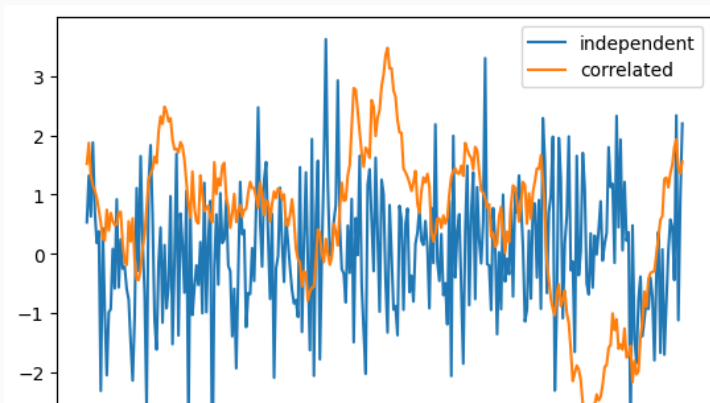
```
def ar1(n, phi):  
    # each value is phi times the previous one, plus fresh noise  
    sigma = (1 - phi ** 2) ** 0.5  
    x = np.zeros(n)  
    x[0] = np.random.normal(0, 1)  
    for i in range(1, n):  
        x[i] = phi * x[i - 1] + np.random.normal(0, sigma)  
    return x
```

```
independent = np.random.normal(0, 1, 3000)  
correlated = ar1(3000, 0.95)
```

```
_, bins, _ = plt.hist(independent, bins=40, alpha=0.5, density=1, label="indep...  
... (2 more lines)
```

Failure 2: correlated samples — cell 138

```
_ = plt.plot(independent[:300], label="independent")  
_ = plt.plot(correlated[:300], label="correlated")  
_ = plt.xlabel("row number")  
_ = plt.legend()
```



Failure 2: correlated samples — cell 140

```
def autocorrelation(x, max_lag):
    x = np.asarray(x) - np.mean(x)
    denominator = np.sum(x * x)
    result = list()
    for lag in range(max_lag + 1):
        result.append(np.sum(x[:len(x) - lag] * x[lag:])) / denominator
    return result

_ = plt.plot(autocorrelation(independent, 60), marker=".", label="independent"...
_ = plt.plot(autocorrelation(correlated, 60), marker=".", label="correlated")
_ = plt.plot(0.95 ** np.arange(61), "k--", label="prediction: 0.95 ** k")
_ = plt.axhline(0, color="k", lw=0.5)
_ = plt.xlabel("lag")
... (2 more lines)
```

Failure 3: heavy tails

- Cauchy: the variance is infinite, and the mean of n draws is no better than one draw
- Averaging does not help, ever
- The running mean jumps when a new extreme arrives
- Same failure hides in an instrument with a bad resolution function

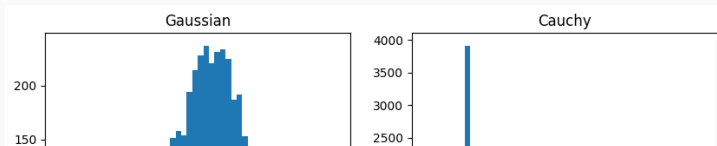
→ cells 146, 152, 157

Failure 3: heavy tails — cell 146

```
gaussian_sample = np.random.normal(0, 1, 4000)
cauchy_sample = np.random.standard_cauchy(4000)

fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].hist(gaussian_sample, bins=50)
axes[0].set_title("Gaussian")
axes[1].hist(cauchy_sample, bins=50)
axes[1].set_title("Cauchy")
plt.show()

print("Cauchy min and max:", cauchy_sample.min(), cauchy_sample.max())
```



Failure 3: heavy tails — cell 152

```
def running_mean(x):  
    x = np.asarray(x)  
    return np.cumsum(x) / np.arange(1, len(x) + 1)  
  
_ = plt.plot(running_mean(gaussian_sample), label="Gaussian")  
_ = plt.plot(running_mean(cauchy_sample), label="Cauchy")  
_ = plt.xlabel("number of points")  
_ = plt.ylabel("running mean")  
_ = plt.legend()
```



Failure 3: heavy tails — cell 157

```
true_value = 10.0
n = 5000
instruments = {"narrow Gaussian noise": lambda: np.random.normal(0, 0.2, n),
               "wide Gaussian noise": lambda: np.random.normal(0, 4.0, n),
               "narrow Cauchy noise": lambda: np.random.standard_cauchy(n) * 0...

# five independent runs of each instrument, measuring the same true value
fig, axes = plt.subplots(1, 3, figsize=(15, 4), sharey=True)
for ax, (label, noise) in zip(axes, instruments.items()):
    for run in range(5):
        ax.plot(running_mean(true_value + noise()), lw=1)
    ax.axhline(true_value, color="k", lw=0.5)
    ax.set_title(label)
    ax.set_xlabel("number of measurements")
... (2 more lines)
```

Failure 4: products

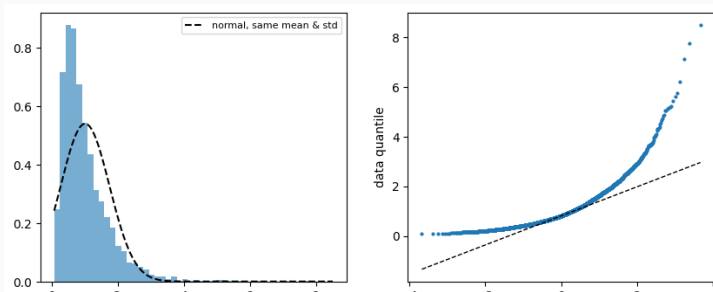
- Gains that multiply, not add — 15 stages of $\pm 30\%$
- Right-skewed, long tail: a log-normal
- Take the log and the product becomes a sum. The CLT was right all along, on the wrong scale
- chol is the real-data version: skew $1.14 \rightarrow 0.09$ under a log

→ cells 160, 163, 165

Failure 4: products — cell 160

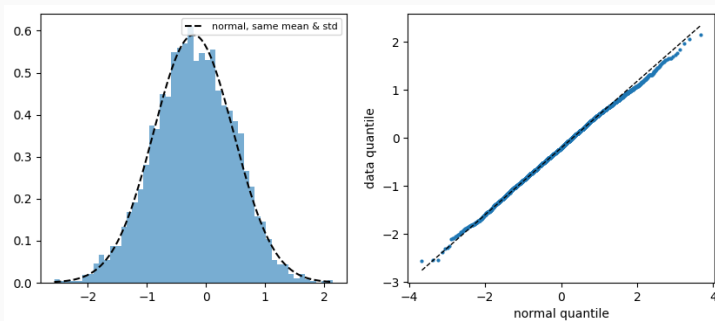
```
stage_gains = np.random.uniform(0.7, 1.3, size=(4000, 15))  
total_gain = np.prod(stage_gains, axis=1)
```

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))  
hist_with_normal(total_gain, ax=axes[0])  
qq_plot(total_gain, ax=axes[1])  
plt.show()
```



Failure 4: products — cell 163

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))  
hist_with_normal(np.log(total_gain), ax=axes[0])  
qq_plot(np.log(total_gain), ax=axes[1])  
plt.show()
```



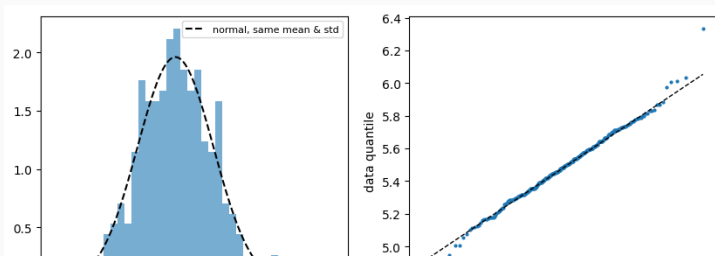
\begin{center}

\end{center}

Failure 4: products — cell 165

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
hist_with_normal(np.log(df["chol"]), bins=40, ax=axes[0])
axes[0].set_xlabel("log(chol)")
qq_plot(np.log(df["chol"]), ax=axes[1])
plt.show()

print("skewness of chol      :", stats.skew(df["chol"]))
print("skewness of log(chol):", stats.skew(np.log(df["chol"])))
```



Not every non-normal histogram is a failure

- oldpeak piles up at zero: bounded, discrete, zero-inflated
- Nothing broke. It simply isn't a sum
- Ask whether the variable is a sum of anything *before* diagnosing it

→ cell 167

Not every non-normal histogram is a failure — cell 167

```
compare_distributions(df, "oldpeak",  
                    {"no disease": df["target"] == 0, "disease": df["target"] == 1},  
                    bins=25, alpha=0.5, density=1)  
plt.xlabel("oldpeak")  
print("fraction exactly zero:", np.mean(df["oldpeak"] == 0))
```



One panel, six views

- Histogram vs normal · Q-Q · log · running mean · autocorrelation · split by covariate
- Top row tests **shape**, bottom row tests **order**
- Run it on `thalach` and it flags the file itself: `heart.csv` is sorted by the label

→ cell 169

One panel, six views — cell 169

```
def diagnose(x, group=None, name=""):
    x = np.asarray(x)
    fig, axes = plt.subplots(2, 3, figsize=(15, 8))
    fig.suptitle(name)

    hist_with_normal(x, ax=axes[0, 0])
    axes[0, 0].set_title("histogram vs normal")

    qq_plot(x, ax=axes[0, 1])
    axes[0, 1].set_title("Q-Q plot")

    if np.all(x > 0):
        hist_with_normal(np.log(x), ax=axes[0, 2])
        axes[0, 2].set_title("histogram of log(x)")
    ... (22 more lines)
```

Lab 6: diagnose it yourself

- Five datasets. Four broken one way each; **one is fine**
- Each comes with a covariate that is real for exactly one of them
- Row order matters — do not sort
- Name the assumption, and the panel that gave it away
- One of them passes every test of shape

Part 3 — Basic data manipulation

A different kind of table

- One row per state. No row is an observation — each is a summary of thousands of cases
- This is the third level of data from the start of the lecture
- Bar plots and pie charts live here: they are a readable form of a *table*
- Not at the start of an analysis. At the end
- The question: do states that inspect workplaces more often have fewer deaths?

→ cells 176, 177

A different kind of table — cell 176

```
!ls -lh
```

```
total 304
```

```
-rw-r--r--@ 1 afarbin  staff    4.8K Oct  5 12:04 2012_Workplace_Fatalities_by_State....  
-rw-r--r--@ 1 afarbin  staff   130K Oct  5 12:04 Lecture.13.ipynb  
-rw-r--r--@ 1 afarbin  staff    11K Oct  5 12:04 heart.csv  
drwxr-xr-x@ 3 afarbin  staff    96B Oct  5 12:03 jupyter
```

A different kind of table — cell 177

```
!head 2012_Workplace_Fatalities_by_State.csv
```

```
State,"Number of Fatalities, 2012","Rate of Fatalities, 2012","State Rank, Fatalitie...  
"South Carolina  
(33.99882060100049, -81.04536765699964)",63,3.5,25,36200,3,492,49,24,111,State  
"West Virginia  
(38.665511497000466, -80.71263935099967)",49,6.9,46,19800,4.1,1798,27,7,173,Federal  
"Massachusetts  
(42.27687306500047, -72.08268985899963)",44,1.4,1,69700,3.1,1929,21,33,123,Federal  
"Tennessee  
(35.680943063000484, -85.77448642199965)",101,3.8,30,65100,3.5,727,45,30,82,State  
... (1 more lines)
```

The file needs repair

- Fields that hold two things at once: a name, a newline, a coordinate pair
- The last four rows are not states — blanks and a national total
- Everything below works on rows 0–49. Those four rows would corrupt every maximum and every average
- Splitting strings into columns is an unglamorous, large share of any real analysis

→ cells 180, 192

The file needs repair — cell 180

df

```
                                State \
0  South Carolina\n(33.99882060100049, -81.045367...
1  West Virginia\n(38.665511497000466, -80.712639...
2  Massachusetts\n(42.27687306500047, -72.0826898...
3  Tennessee\n(35.680943063000484, -85.7744864219...
4  Oklahoma\n(35.472034350000456, -97.52106845499...
5  Illinois\n(40.48501278700047, -88.99770813999965)
6  Nebraska\n(41.64104043900045, -99.36571864599966)
7  Delaware\n(39.00883351400046, -75.57773943699965)
... (326 more lines)
```

The file needs repair — cell 192

```
# Clean it up
```

```
state_names=list(map(lambda x: x.split("\n")[0],df[df.columns[0]][0:50]))
```

```
state_names
```

```
['South Carolina',  
 'West Virginia',  
 'Massachusetts',  
 'Tennessee',  
 'Oklahoma',  
 'Illinois',  
 'Nebraska',  
 'Delaware',  
 'Hawaii',  
 ... (41 more lines)]
```

Which state is worst?

- Max, then **argmax**, then use the position to look up the name
- The answer is Texas. The answer is useless
- More workers, more fatalities — the count ranks states by size
- The comparable quantity is a rate. Same normalization argument as the histograms

→ cells 185, 187, 198

Which state is worst? — cell 185

```
# Max of a column  
# Note that the last 4 rows are not states  
np.max(df[df.columns[1]][0:50])  
  
np.float64(536.0)
```

Which state is worst? — cell 187

```
# What was the state?
```

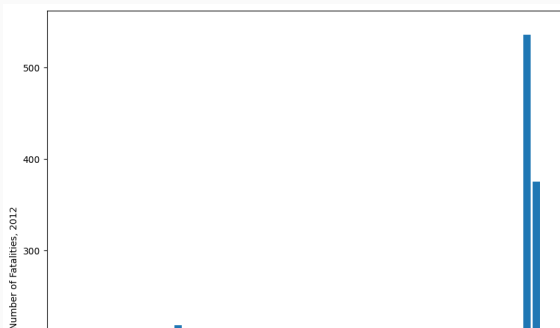
```
df[df.columns[0]][np.argmax(df[df.columns[1]][0:50])]
```

```
'Texas\n(31.827243635000457, -99.4267664729997)'
```

Which state is worst? — cell 198

```
fig,ax= plt.subplots(1,1,figsize=(10,10))

ax.bar(range(50),df[df.columns[1]][0:50])
ax.set_xticks(range(50))
_=ax.set_xticklabels(state_names, rotation='vertical', fontsize=10)
_=ax.set_ylabel(df.columns[1])
```



Count, rate, and a hypothesis

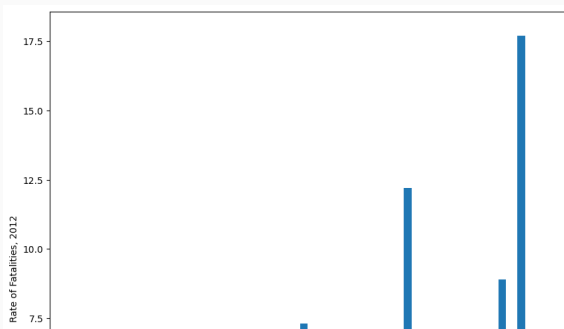
- By rate the ranking changes completely: small states top it, North Dakota first
- A rate from a small workforce is built from few deaths — $\sqrt{n}$ again
- Inspection interval: *years* between visits to a workplace. Best state ~31, worst over 500
- Far fewer inspectors than workplaces, everywhere

→ cells 200, 202

Count, rate, and a hypothesis — cell 200

```
fig,ax= plt.subplots(1,1,figsize=(10,10))

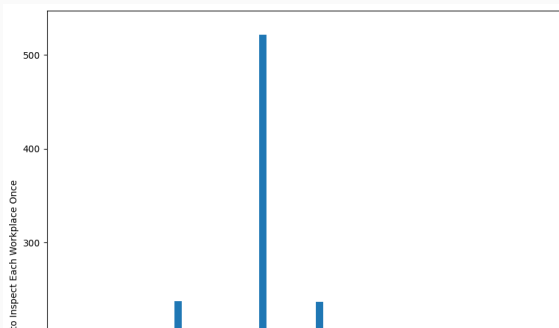
ax.bar(range(50),df[df.columns[2]][0:50])
ax.set_xticks(range(50))
_=ax.set_xticklabels(state_names, rotation='vertical', fontsize=10)
_=ax.set_ylabel(df.columns[2])
```



Count, rate, and a hypothesis — cell 202

```
fig,ax= plt.subplots(1,1,figsize=(10,10))

ax.bar(range(50),df[df.columns[9]][0:50])
ax.set_xticks(range(50))
_=ax.set_xticklabels(state_names, rotation='vertical', fontsize=10)
_=ax.set_ylabel(df.columns[9])
```



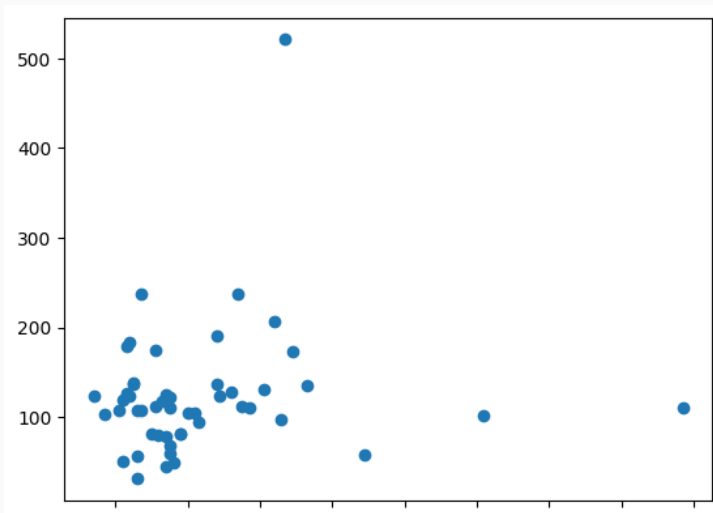
Does enforcement show up?

- The prediction: longer interval $\rightarrow$ higher fatality rate, trending upward
- It does not. The one state above 500 years sits mid-range
- The ratio plot asks the same question in different coordinates
- A negative result is a result

$\rightarrow$ cells 204, 206

Does enforcement show up? — cell 204

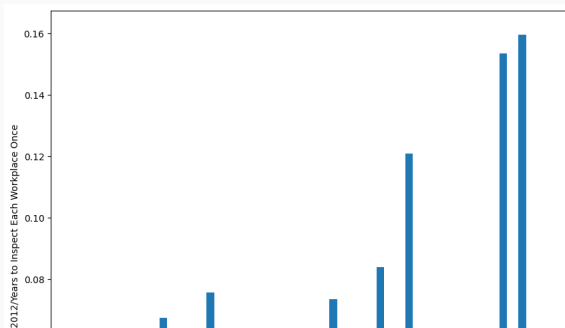
```
plt.scatter(df[df.columns[2]][0:50],df[df.columns[9]][0:50])
```



Does enforcement show up? — cell 206

```
fig,ax= plt.subplots(1,1,figsize=(10,10))

ax.bar(range(50),df[df.columns[2]][0:50]/df[df.columns[9]][0:50])
ax.set_xticks(range(50))
_=ax.set_xticklabels(state_names, rotation='vertical', fontsize=10)
_=ax.set_ylabel(df.columns[2]+"-"+df.columns[9])
```



Where looking stops

- Exploring is a loop: histogram, notice, select, look at the other columns, find nothing, go back
- Every plot is made for a reader, and the first reader is you
- Putting a number on “how strong” and “how likely by chance” is a statistics course
- A model’s accuracy on a dataset nobody has looked at should not be believed